

# CleeVoice

Dictado por voz para todo tu sistema operativo

*Una tecla. Habla. Aparece texto.*

Documentación maestra del proyecto

Construido para JoinsClee · Open-source · Multiplataforma

Versión 1.0 · Mayo 2026

## Índice

1. Naming — ¿Por qué CleeVoice?
2. ¿Qué estamos construyendo?
3. Arquitectura técnica
4. Flujo de usuario
5. Roadmap por fases
6. Costos esperados
7. Riesgos y mitigaciones
8. Estructura del repositorio
9. Cómo empezar con Claude Code
10. Referencias open-source
11. Prompt maestro para Claude Code
12. Resumen ejecutivo

## 1. Naming — ¿Por qué CleeVoice?

Antes de empezar a construir, fijamos el nombre. Cinco opciones apalancadas en el ecosistema JoinsClee, ordenadas de más recomendada a alternativas:

|             |          |                                                                                    |                                  |
|-------------|----------|------------------------------------------------------------------------------------|----------------------------------|
|             |          |                                                                                    |                                  |
| CleeVoice ★ | clí-vois | Claro, directo, dice qué hace. Mantiene la raíz Clee. Dominio probablemente libre. | Algo descriptivo, poco "marca"   |
| Cleeko      | clí-ko   | Corto, brandeable, suena tech. Fácil de pronunciar en ES e inglés.                 | No dice qué hace por sí solo     |
| Cleespeak   | clí-spik | Descriptivo. Suena premium.                                                        | Más largo                        |
| Vooclee     | vu-clí   | Mezcla "voice" + "Clee". Memorable.                                                | Pronunciación dudosa fuera de ES |
| Sayclee     | sei-clí  | Verbo + marca. Sentido publicitario.                                               | Suena más a feature que a app    |

**Recomendación:** usar **CleeVoice** como nombre del producto y [cleevoice](#) como slug técnico (carpeta, repo, app id, dominio). Es el más claro para que cualquier cliente lo entienda al primer vistazo, y el más fácil de SEO-ear si algún día abrimos un landing en [cleevoice.joinsclee.com](#).

## 2. ¿Qué estamos construyendo?

CleeVoice es una app de dictado por voz que vive en segundo plano. Funciona así:

- Está corriendo en la bandeja del sistema (tray icon).
- El usuario presiona un atajo global (Ctrl+Shift+Espacio en Windows / Cmd+Shift+Espacio en Mac) estando en cualquier app — Gmail, Notion, ChatGPT, VSCode, Skool, GoHighLevel, Slack, Word, etc.
- Aparece un mini-overlay flotante "🎤 Escuchando..." mientras habla.
- Suelta la tecla (o vuelve a presionar el atajo).
- La app transcribe localmente con Whisper, limpia el texto con un LLM, y lo pega automáticamente donde estaba el cursor.

Resultado: el usuario habla y el texto aparece. Punto. No abre otra ventana, no copia-peg, no cambia de contexto.

## Diferenciales vs. Glaido

|                |                       |                                 |
|----------------|-----------------------|---------------------------------|
|                |                       |                                 |
| Plataforma     | Solo macOS            | Windows + Mac desde día 1       |
| Stack          | App nativa Mac        | Electron multiplataforma        |
| Modelo         | Cloud propietario     | Local (gratis) + Cloud opcional |
| Privacidad     | Procesa en su nube    | 100% local por defecto          |
| Costo          | Freemium (Pro pagado) | Gratis (modo local)             |
| Idiomas        | 100+                  | 100+ (Whisper soporta todo)     |
| Custom prompts | Sí                    | Sí, conectado al Método CLEE    |

Por qué esto tiene sentido para JoinsClee

Tres razones de negocio:

- **Productividad interna del equipo.** Cristhian + setters + media-buyers + redactores escriben cientos de mensajes, copys, scripts y respuestas a comunidad cada día. Una app de dictado bien hecha les devuelve 1-2 horas diarias por persona.
- **Demo de "AI-first agency".** Poder mostrarle a un cliente potencial "esta app la construimos nosotros con Claude Code en una semana" refuerza la PUV de la agencia.
- **Posible producto/lead magnet futuro.** Descarga gratuita con branding JoinsClee, captura email, upgrade a Cloud con LLM premium. MiniApp pero a escala de desktop.

3. Arquitectura técnica

Stack elegido

|                                        |  |
|----------------------------------------|--|
| CleeVoice App                          |  |
| Electron 33 — Node 24 + Chromium       |  |
| UI: React 19 + Tailwind + shadcn/ui    |  |
| └─ Tray icon                           |  |
| └─ Recording overlay (siempre encima)  |  |
| └─ Settings window                     |  |
| Main process (Node)                    |  |
| └─ globalShortcut → captura hotkey     |  |
| └─ Audio capture (Web Audio API)       |  |
| └─ Audio buffer → WAV                  |  |
| └─ whisper.cpp local / Groq cloud      |  |
| └─ Post-procesamiento LLM (limpieza)   |  |
| └─ Inyección texto (clipboard + paste) |  |
| Local storage                          |  |
| └─ better-sqlite3 → historial          |  |
| └─ electron-store → settings           |  |
| └─ Modelos en ~/.cleevoice/models/     |  |

Decisiones clave y por qué

|                     |                                                                                                                                                                     |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Electron (no Tauri) | Code-share Win+Mac, ecosistema npm completo, ejemplos abiertos (OpenWhispr, Whispering). Tauri es más liviano pero menos maduro para audio + globalShortcut + tray. |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                   |                                                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| whisper.cpp local (no Python)     | Inferencia rápida en CPU, sin instalar Python, binario empaquetable. Mismo binario funciona en Mac (Metal) y Windows. |
| Modelo base (~140 MB) por default | Equilibrio precio/calidad. En español da transcripción decente en ~1-2s para clips de 10s.                            |
| Groq como cloud fallback gratis   | Groq sirve Whisper-large-v3-turbo gratis con rate limit generoso. Es el plan B "LLM gratuito" con calidad cloud.      |
| Paste sintético (no API)          | Copiamos al clipboard, simulamos Cmd/Ctrl+V. Funciona universalmente sin permisos por app.                            |
| Limpieza con LLM opcional         | Groq Llama 3.3 70B gratis. Off por defecto para evitar latencia; el usuario lo prende.                                |

## Stack final resumido

- Runtime: Node.js 24, Electron 33
- UI: React 19, TypeScript, Tailwind CSS v4, shadcn/ui
- Audio: Web Audio API → WAV buffer
- STT local: whisper.cpp (modelos base/small/medium descargables)
- STT cloud: Groq Whisper-large-v3-turbo (gratis)
- LLM cleanup: Groq Llama 3.3 70B (gratis) — opcional
- DB: better-sqlite3 (historial local)
- Settings: electron-store con cifrado safeStorage
- Build: electron-builder (NSIS Windows, DMG Mac)
- Hotkey: globalShortcut + node-global-key-listener (push-to-talk)
- Paste: @nut-tree-fork/nut-js (Win) + AppleScript (Mac)

## 4. Flujo de usuario completo

1. Usuario instala CleeVoice.exe / CleeVoice.dmg

2. Primer arranque → wizard de 3 pasos:

- a) Permisos de micrófono y accesibilidad
- b) Elige modelo (base recomendado / cloud Groq con API key)
- c) Elige hotkey (default Ctrl+Shift+Espacio)

3. App va a tray. Listo.

— En uso —

4. Usuario está en Gmail escribiendo un correo.

5. Presiona Ctrl+Shift+Espacio (push-to-talk).

→ aparece overlay "🎤".

- 6. Habla: "Hola Camilo, gracias por tu mensaje de ayer, mañana te mando la propuesta actualizada"
- 7. Suelta la tecla.  
→ overlay muestra "🌀 procesando..."
- 8. ~1.5s después → texto aparece pegado en Gmail, capitalizado y con puntuación.
- 9. Overlay desaparece. Usuario sigue escribiendo.

## 5. Roadmap por fases

Pensado para construir con Claude Code de forma incremental. Cada fase es un PR funcional. Regla de oro: no avanzar sin que la fase anterior pase el criterio de éxito.

|                            |                                                             |     |
|----------------------------|-------------------------------------------------------------|-----|
|                            |                                                             |     |
| 0. Setup                   | Electron + React + TS arrancando, "Hello CleeVoice" visible | 4h  |
| 1. Tray + Hotkey + Overlay | App vive en tray. Hotkey global muestra overlay temporal    | 8h  |
| 2. Captura de audio        | El hotkey graba audio del mic y guarda un WAV               | 6h  |
| 3. Whisper local           | El WAV se transcribe automáticamente con whisper.cpp        | 12h |
| 4. Inyección de texto      | El texto transcrito se pega donde está el cursor            | 4h  |
| 5. Settings UI             | Ventana de configuración funcional con todas las tabs       | 8h  |
| 6. Groq cloud fallback     | Modo cloud con Groq como alternativa más rápida             | 6h  |
| 7. Limpieza con LLM        | Texto pasa por Llama 3.3 para limpiar muletillas            | 6h  |
| 8. Historial + diccionario | Persistir transcripciones, aplicar diccionario JoinsClee    | 6h  |
| 9. Pulido y empaquetado    | Instaladores firmados listos para distribuir                | 12h |
| TOTAL                      |                                                             | 72h |

Estimación: a 6-7h productivas por día con Claude Code asistiendo = ~10-11 días hábiles. Forkeando OpenWhispr y rebrand = ~5-6 días.

### Detalle por fase

## **Fase 0 — Setup (4h)**

npm init, electron-vite con template react-ts, Tailwind v4 + shadcn/ui, estructura de carpetas, scripts npm run dev y build. Validación: corres dev, abre ventana con "CleeVoice — Listo para construir".

## **Fase 1 — Tray + Hotkey + Overlay (8h)**

Tray icon con menú, globalShortcut.register, ventana overlay con frame:false transparent:true alwaysOnTop:true. Al disparar hotkey aparece overlay flotante 280×100 centrado abajo. Cerrar ventana ≠ cerrar app. Validación: funciona en Win y Mac estando en cualquier app.

## **Fase 2 — Audio (6h)**

MediaRecorder en renderer, IPC al main, conversión webm→wav 16kHz mono con ffmpeg-static, guarda en /tmp. Validación: WAV reproducible con el dictado correcto.

## **Fase 3 — Whisper local (12h)**

Bundle de binarios whisper-cli, descarga del modelo ggml-base.bin en primer uso (~140MB con progreso), wrapper con child\_process.spawn, integración con flow. Validación: 4 de 5 frases en español se transcriben correctamente en <3s.

## **Fase 4 — Inyección de texto (4h)**

Clipboard + AppleScript en Mac, @nut-tree-fork/nut-js en Win. Restaura clipboard previo después de 1s. Onboarding para permiso de accesibilidad Mac. Validación: texto aparece en Gmail, Notepad, VSCode, ChatGPT, Cursor.

## **Fase 5 — Settings UI (8h)**

Ventana con 4 tabs: General, Modelo (download manager), Atajos (re-registro live), Diccionario (precargado con términos JoinsClee). Persiste con electron-store. Validación: cambio hotkey → reinicia → persiste.

## **Fase 6 — Groq cloud (6h)**

Tab Cloud con API key cifrada vía safeStorage. Cliente Groq con whisper-large-v3-turbo. Router de engine local/cloud. Fallback automático si red falla. Validación: dictado <1s con calidad mejor.

## **Fase 7 — LLM cleanup (6h)**

active-win detecta app activa → contexto dinámico. Llama 3.3 70B vía Groq con prompt configurable. Toggle on/off. Validación: "eh este o sea quería decirte" → "Quería decirte".

## **Fase 8 — Historial + diccionario (6h)**

SQLite con better-sqlite3, tab Historial buscable, diccionario aplicado via --prompt de whisper + post-procesamiento regex. Validación: "skul" → "Skool", "hormosí" → "Hormozi".

## **Fase 9 — Pulido y empaquetado (12h)**

Auto-updater electron-updater, branding completo, onboarding 5 pantallas, DMG firmado y notariado,

NSIS Windows, landing en cleevoice.joinsclee.com. Validación: usuario nuevo en <3min dicta su primera frase.

## 6. Costos esperados

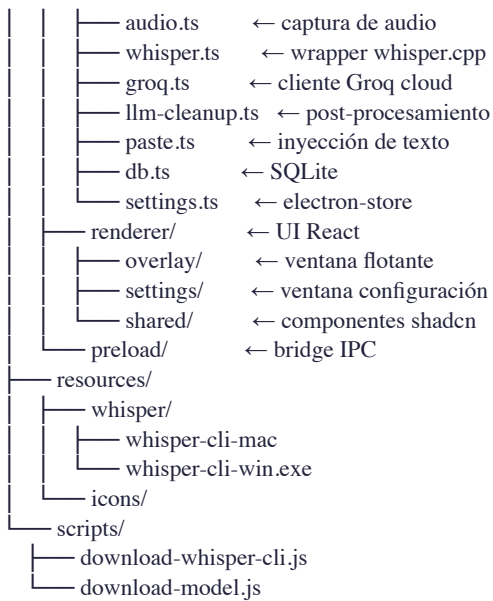
|                             |                                            |
|-----------------------------|--------------------------------------------|
|                             |                                            |
| Modelo local Whisper        | \$0 (corre en CPU del usuario)             |
| Groq Whisper cloud          | \$0 hasta ~14,400 segundos/día por API key |
| Groq Llama 3.3 70B          | \$0 hasta ~6,000 requests/día              |
| Apple Developer (firma Mac) | \$99/año (opcional pero recomendado)       |
| Windows code signing cert   | \$200-400/año (opcional)                   |
| Hosting landing (Vercel/GH) | \$0                                        |
| TOTAL mínimo funcional      | \$0                                        |
| TOTAL "presentable" firmado | \$99-499                                   |

## 7. Riesgos y mitigaciones

|                                                |                                                                                                        |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------|
|                                                |                                                                                                        |
| Calidad de Whisper base en español no convence | Plan B desde día 1: Groq cloud con large-v3-turbo gratis es la opción más rápida y precisa que existe. |
| Inyección falla en alguna app específica       | Fallback: copia al clipboard y muestra notificación. El usuario nunca queda colgado.                   |
| Permisos en macOS confunden                    | Wizard de onboarding con screenshots paso a paso. OpenWhispr ya resolvió esto bien.                    |
| Modelos pesan mucho                            | Descarga bajo demanda al primer uso, no se empaqueta. Instalador queda ~80MB.                          |
| Hotkey colisiona con otra app                  | Settings permite cambiarlo. Si la registración falla, mostrar warning.                                 |

## 8. Estructura del repositorio

```
cleevoice/
├── README.md
├── PROMPT_MASTER_CLAUDE_CODE.md
├── ARCHITECTURE.md
├── ROADMAP.md
├── package.json
├── electron-builder.yml
├── src/
│   ├── main/           ← Electron main process
│   │   ├── index.ts    ← entry point
│   │   ├── tray.ts     ← tray icon + menu
│   │   └── hotkey.ts    ← globalShortcut
```



## 9. Cómo empezar con Claude Code

Tres archivos en este paquete:

- **README.md** — contexto completo del proyecto.
- **ARCHITECTURE.md** — detalle técnico de cada módulo.
- **PROMPT\_MASTER\_CLAUDE\_CODE.md** — el prompt que copias en Claude Code para arrancar Fase 0. Cada fase tiene su propio sub-prompt continuador.

Flujo recomendado:

# 1. En tu Mac/PC

```
mkdir cleevoice && cd cleevoice
git init
```

# 2. Abre Claude Code en esta carpeta

```
claude
```

# 3. Pega el contenido de

```
# PROMPT_MASTER_CLAUDE_CODE.md
```

# 4. Claude Code construye Fase 0 + Fase 1

# 5. Pruebas, commit, pasas al siguiente

```
# prompt de fase
```

Cada fase es un commit. No avances a la siguiente sin probar la anterior. Eso te asegura que en cualquier punto tienes una app que funciona.



## 10. Referencias open-source

Estos proyectos ya resolvieron mucho de lo que vamos a construir. Leerlos antes de codear ahorra días:

- **OpenWhispr** — [github.com/OpenWhispr/openwhispr](https://github.com/OpenWhispr/openwhispr) — Electron + React + whisper.cpp, MIT, multiplataforma. *Es lo más cercano a CleeVoice. Estudiar su estructura.*
- **Whispering** — [github.com/braden-w/whispering](https://github.com/braden-w/whispering) — Tauri (alternativa más liviana), buen UX, también MIT.
- **whisper.cpp** — [github.com/ggerganov/whisper.cpp](https://github.com/ggerganov/whisper.cpp) — el motor de inferencia local.
- **faster-whisper** — [github.com/SYSTRAN/faster-whisper](https://github.com/SYSTRAN/faster-whisper) — alternativa Python si quisiéramos backend separado.
- **node-global-key-listener** — [github.com/LaunchMenu/node-global-key-listener](https://github.com/LaunchMenu/node-global-key-listener) — para push-to-talk verdadero.

### Estrategia inteligente: fork OpenWhispr

En lugar de construir desde cero: forkear OpenWhispr → rebrand a CleeVoice → quitar lo que no usamos (meeting transcription, agente, calendar) → ajustar prompts y diccionario al ecosistema JoinsClee. **Esto recorta el roadmap de 10 días a ~4-5 días.**

Esta decisión está abierta. El roadmap de 10 días asume construcción desde cero (más control). El fork asume velocidad máxima. Recomendación: empezar desde cero las primeras 2 fases para entender bien la arquitectura, y si en Fase 3 sientes que vas lento, evaluar forkear el módulo de whisper de OpenWhispr.

## 11. Prompt maestro para Claude Code

Este es el bloque que copias literalmente en tu primera sesión de Claude Code dentro de la carpeta vacía donde quieres construir CleeVoice. El prompt completo con todos los prompts por fase está en el archivo PROMPT\_MASTER\_CLAUDE\_CODE.md adjunto.

### Prompt de arranque (Fase 0 + Fase 1)

Vamos a construir CleeVoice, una app de dictado por voz multiplataforma (Windows + Mac) inspirada en Glaido pero open-source y con modelo local gratuito por defecto.

#### CONTEXTO DEL PROYECTO

CleeVoice vive en la bandeja del sistema. El usuario presiona un atajo global (Ctrl+Shift+Espacio), habla en cualquier app, y el texto transcrito aparece pegado donde estaba el cursor. Procesamiento local con whisper.cpp (gratis, privado) y opción cloud con Groq (también gratis con API key).

Este proyecto es para JoinsClee, agencia de marketing especializada en lanzamientos digitales y comunidades Skool.

Servirá como herramienta interna de productividad y posiblemente como producto público.

STACK OBLIGATORIO

- Electron 33+ con electron-vite
- React 19 + TypeScript estricto
- Tailwind CSS v4 + shadcn/ui
- whisper.cpp para transcripción local
- Groq SDK para cloud (whisper-large-v3-turbo + llama-3.3-70b-versatile)
- better-sqlite3 para historial
- electron-store para settings
- @nut-tree/fork/nut-js para inyección Windows
- electron-builder para empaquetado

TU PRIMERA TAREA: Fase 0 + Fase 1  
[continúa en PROMPT\_MASTER\_CLAUDE\_CODE.md...]

Reglas universales del proyecto

- No ejecutar npm install con dependencias fuera de la lista permitida sin avisar.
- No inventar APIs ni nombres de paquetes. Si dudas, busca primero.
- No usar any en TypeScript salvo que se justifique.
- Después de cada cambio significativo, correr npm run dev y verificar.
- Al terminar una fase, listar archivos cambiados y esperar visto bueno.
- Comentarios en español cuando expliquen lógica de negocio. Variables y funciones en inglés.
- Si algo no funciona en una plataforma, decirlo explícitamente.
- Para empaquetado y firma, nunca inventar paths; verificar con doc oficial.

12. Resumen ejecutivo

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
|                   |                                                                                       |
| Qué               | App de dictado por voz multiplataforma para JoinsClee y posiblemente producto público |
| Nombre            | CleeVoice (alternativas: Cleeko, Cleespeak)                                           |
| Stack             | Electron + React + whisper.cpp + Groq (cloud gratis)                                  |
| Costo Y1          | \$0 funcional / \$99-499 firmado                                                      |
| Tiempo desde cero | 10-11 días con Claude Code                                                            |
| Tiempo forkeando  | 4-5 días forkeando OpenWhispr                                                         |
| Próximo paso      | Abrir Claude Code en carpeta vacía y pegar PROMPT_MASTER_CLAUDE_CODE.md               |

Próximos pasos concretos

- **Hoy:** decidir naming final (recomendado: CleeVoice). Comprar dominio cleevoice.com o usar

subdominio en joinsclee.com.

- **Esta semana:** crear carpeta del proyecto, copiar los 4 archivos (README, ARCHITECTURE, ROADMAP, PROMPT\_MASTER) y arrancar Claude Code con Fase 0.
- **Semana 1:** completar Fases 0-3 (setup, tray, hotkey, audio, transcripción local). App funcionalmente probada en Mac y Windows.
- **Semana 2:** completar Fases 4-7 (inyección texto, settings UI, Groq cloud, LLM cleanup). Producto usable por Cristhian a diario.
- **Semana 3:** completar Fases 8-9 (historial, diccionario, empaquetado). Instaladores firmados disponibles para el equipo JoinsClee.
- **Post-launch:** recoger feedback de 5-10 usuarios internos durante 2 semanas. Iterar. Decidir si abrir como producto público.